VQL Language Server — Design Notes & Decisions

Living document tracking the design, decisions, and considerations for building an LSP server for VQL. Update as the work progresses.

Last updated: 2026-08-04

Status: diagnostics milestone complete. The whole-document parse-failure problem is mitigated (truncate-and-retry, see Hurdles) and the artifact registry is implemented (see Artifact store). Pull-based diagnostics (LSP 3.17 `textDocument/diagnostic`) are implemented for opencode compatibility. All five agent-discovery options in VQL-LSP-USAGE.md are implemented and verified end-to-end (CLI, skill, custom opencode tool, mcpls bridge, native MCP tool). Document symbols (`textDocument/documentSymbol`) are implemented and verified end-to-end (see Foundation Work and Current State). Hover (`textDocument/hover`) and completion (`textDocument/completion`) are implemented and verified end-to-end (see Current State). Go-to-definition and validating artifact parameters against artifact defaults are possible future extensions, not planned work. The `lsp` command now runs in **hybrid mode:** it boots a local gRPC API server (like the `gui` command) and connects to it as the superuser, so the registry is built from the *repository* — including custom artifacts stored in the datastore — rather than only the built-in scope (see "Decision: hybrid mode").

Goal

Create a Language Server Protocol (LSP) server for VQL that can be used by AI agents (e.g. OpenCode, Claude Code) when they need to formulate VQL queries.

Rationale (see https://amirteymoori.com/lsp-language-server-protocol-ai-coding-tools/): LSP gives AI tools *semantic* understanding of code instead of text matching. For VQL this means the agent gets precise diagnostics, plugin lookup, and type information — the difference between an agent that guesses and one that knows.

Target Use Cases

1. **Primary:** An AI agent creates a pure VQL query and submits it to a Velociraptor server via the API query endpoint. The LSP validates the query *before* submission, catching hallucinated plugin names, bad argument names, wrong types, and undefined LET variables.
2. **Secondary:** An AI agent creates a query and hands it to the user, who decides where to run it (a notebook cell, embedded in a YAML artifact, etc.).

Key implication: **the LSP is NOT file-based.** The "document" is a single VQL query string, passed to the server as a virtual document (e.g. `untitled:` URI). No workspace folders, no file watching, no YAML extraction, no incremental change tracking.

Byproduct, not a use case: Because the server speaks standard LSP it is technically client-agnostic, so a thin editor extension (~100 lines) could wire `velociraptor lsp` into VS Code, Neovim, Zed, etc. and get squiggles, hover docs, autocomplete and an outline for `.vql` files. That is a natural *byproduct* of using a standard protocol — not an intended use case. The point of this project is the agent pre-flight validation loop, not a human IDE experience. Documented in VQL-LSP-USAGE.md as Option 4 (future). One caveat if anyone ever builds it: our positions are byte-based (fine for ASCII VQL) while editors like VS Code use UTF-16 character offsets, so a position-conversion shim would be needed for non-ASCII text.

Feature Priority (agent-first)

1. **Diagnostics** — syntax errors (with precise line/column), unknown plugins/functions, unknown argument names, argument type mismatches, undefined LET variable references.
2. **Hover** — signature + docs + argument descriptions for a plugin or function under the cursor, so the agent can self-serve API knowledge.
3. **Document symbols / Go to definition** — LET variables, artifact references, query structure. **Implemented** (see Current State).
4. **Completion** — mostly useful for the human in the loop; lowest priority for the agent use case.

Architecture Decisions

Decision: LSP server lives in the Velociraptor binary

The server is a new command in the Velociraptor binary:

```
velociraptor lsp
```

Why: the server needs every plugin, function, and artifact name registered in the server's VQL scope, and that registry only exists inside the full Velociraptor build. A standalone repo would have to import half of Velociraptor anyway and would go stale every time a plugin is added.

Decision: Hybrid mode — the `lsp` command is a local API client

One design approach considered was to make the LSP an *API endpoint* so it could work directly with internal services (repository manager etc.) and see custom artifacts in addition to built-ins. After analysis we landed on a hybrid that gets the same benefit without changing the API surface:

- The `lsp` command bootstraps like the `gui`/`collector` commands: it accepts an optional `--datastore` flag, defaults to a temp datastore (`gui_datastore`), generates a fresh `server.config.yaml` + `client.config.yaml` if none exists, and starts `StartToolServices`.
- It then starts a **local gRPC API server** via `api.NewServerBuilder(...)` + `server_builder.WithAPIServer(...)` (the same machinery the GUI uses for its API service), skipping `StartServer` so no frontend/GUI listeners come up.
- It connects to that local server as the **superuser** identity (`grpc_client.Factory.GetAPIClient(ctx, grpc_client.SuperUser, config_obj)`), presenting the generated frontend certificate. No password, no auth ceremony — the server treats a client holding the frontend cert as superuser (`services/users/grpc.go GetUserFromContext`).
- The artifact registry is then built by calling the existing `GetArtifacts` API method over gRPC (`BuildRegistryFromAPIClient`), which pulls from the repository manager — so **custom artifacts stored in the datastore are visible to the LSP**, not just the built-ins in the base scope.

Why this is the right shape:

- **No API change.** The proto, the server, and the RPC surface are all untouched. The team's "API endpoint" idea would have meant adding a new bidi-streaming RPC to the public API; the hybrid achieves the same repository access locally with zero API surface change.
- **Custom artifacts work.** Anyone who puts an artifact YAML in the datastore (or uploads it via the GUI) gets it validated by the LSP immediately.

- **Port-collision avoidance is automatic.** The lsp command tries the default API port (8001); if it is already taken (e.g. a production server on the same machine as the MCP+LSP), it picks a free ephemeral port on the same address. No flags needed.
- **Local deployment modes.** If the agent/MCP/LSP and Velociraptor are on different machines, custom artifacts can be copied into a local "fake" datastore, or the LSP can run on the same machine as the server/datastore (e.g. a production box) where it shares the real datastore. The default temp datastore path means a bare `velociraptor lsp` always works even with no configuration at all. The LSP server deliberately avoids implementing remote API access because that adds too much overhead (configuration, auth+ACLs, potential troubleshooting, etc.) - the overriding goal is to keep the LSP server as simple as possible, plus avoid the potential security risk of encouraging users to copy a file containing API credentials to their workstation just for the convenience of having access to custom artifacts.

Decision: Diagnostics-first implementation strategy

Build the core engine, then prove it with diagnostics alone, then layer on hover/completion.

- The engine = scope builder (all plugins/functions registered) + parser + AST walker + position-to-node mapping.
- Diagnostics is the first handler but forces the whole engine to exist.
- Hover/completion are the same introspection data queried differently — cheap once the engine exists.
- De-risks the trickiest subsystem before building features on top of it.

Decision: LSP library — go.lsp.dev

Use **go.lsp.dev** (`go.lsp.dev/protocol` + `go.lsp.dev/jsonrpc2`) — the de-facto standard for Go LSP servers. Actively maintained, LSP 3.18 types generated from the official meta-model, fast JSON-RPC layer. Embed `UnimplementedServer` and implement the handlers we need. Requires Go 1.26+.

Rejected alternatives:

- **tliron/glsp** — mature SDK, ready-to-run JSON-RPC server supporting stdio/TCP/WebSockets/IPC.
- **LukasParke/gossip** — newer framework with built-in document store, tree-sitter integration, incremental diagnostics.

Decision: Bump minimum Go version to 1.26

Velociraptor's go.mod `go` directive was bumped from `1.25.3` to `1.26.4`.

Why: go.lsp.dev (`go.lsp.dev/protocol` + `go.lsp.dev/jsonrpc2`) requires Go 1.26+, and Go's module resolution bumps the module's `go` directive when a dependency needs a newer version. Velociraptor uses a single main go.mod (no nested modules), so the bump applies to the whole project, not just the LSP command.

Accepted trade-off: raising the project-wide minimum Go build version. The current toolchain already runs 1.26.4, so this works locally with no other changes; it is purely a build-requirement change.

Rejected alternative: pinning an older go.lsp.dev release that still fit Go 1.25, which would forfeit the maintained, current-API version.

Hurdles

Participle is a whole-document parser (the big one)

Participle throws away the entire AST on the first syntax error and returns nil. This kills completions and diagnostics exactly when the user (or agent) needs them most — in the middle of a half-typed query.

Mitigation implemented: truncate-and-retry. The `Validate()` loop in `vql/lsp/diagnostics.go` scans the document segment by segment:

1. Parse the current segment (starting at a byte offset `base`).
2. On success, validate all statements in it and stop.
3. On failure, unwrap the error (`errors.Cause` → `*lexer.Error` or `participle.Error`) to get an absolute byte offset, emit a syntax diagnostic at that position, then recover whatever was parseable *before* the error by retrying progressively shorter prefixes (backing up to the previous `\n` or `;` boundary — `largestParseablePrefix`).
4. Jump past the errored line and continue scanning from the next line.

Result: a garbage line in the middle of a document no longer hides errors on either side of it. A doc with `pslist(foo=1)`, a `@@@` garbage line, `pslist(bar=2)`, and an unknown plugin reports all four problems with exact positions. Retries are capped (`maxParseErrors = 50`).

Positions are mapped from participle's byte offsets back to LSP line/character via a `positionMapper` (binary search over line-start offsets). This works because participle positions are byte-based and VQL documents in the agent use case are ASCII/UTF-8 friendly; multi-byte characters are a known caveat.

Fallback strategies considered but deferred:

- Raw lexer-token fallback when the parse fails (the token stream is available from the lexer) — not needed since truncate-and-retry gives useful coverage.
- Position-aware error mapping — done as part of the above.

Artifacts are not registered in the base scope

`Artifact.*` names only resolve when the artifact repository is loaded into scope, which the base server scope does not do. Solved — see "Artifact store" below.

Foundation Work Already Done

participle v2 migration (branch `participle-v2-upgrade`)

- **vfilter** (`~/projects/vfilter`): migrated to `github.com/alecthomas/participle/v2` `v2.1.4`.
- Lexer rewritten from a single mega-regex to `[]lexer.SimpleRule`.
- Generic `MustBuild[T]`, new `ParseString` signature.
- Comment elision preserved via custom `droppingLexerDef` wrapper.
- Error handling updated for `*lexer.Error` / `participle.Error`.
- Commits: `abfe2d4` (migration), `95a1f79` (position capture).
- **Velociraptor**: migrated its own direct participle usage.
- `vql/windows/wmi/parse/parse.go` (MOF parser) — lexer, MustBuild, ParseString.
- `services/repository/errors.go` — `UnexpectedTokenError` is now a pointer.
- `services/repository/reformat.go` — `participle.Error` now exposes `Position()` instead of `Token()`.
- `go.mod`: v2 as direct dep; v0.7.1 remains only as indirect dep of sigma-go. Local `replace www.velocidex.com/golang/vfilter => ../vfilter` enabled.
- Commit: `c986183e5`.

Source position capture (the LSP enabler)

Added Pos/EndPos lexer.Position fields to key AST nodes in vfilter, auto-populated by participle v2:

- VQL, _Select, Plugin, _Args, _AliasedExpression, _MemberExpression, _AndExpression, _SymbolRef.

Verified spans map exactly to source (e.g. plugin Artifact.Linux.Sys in a SELECT query spans exactly its source offset). This is what makes precise diagnostics possible.

Inspect() API in vfilter (uncommitted on participle-v2-upgrade)

The grammar node types in vfilter are unexported, so an external package cannot name them. Added a small exported inspection API (~/projects/vfilter/inspect.go + inspect_test.go):

- vfilter.Inspect(vql *VQL) *Inspection walks a parsed statement and returns Inspection{Calls []CallInfo, Symbols []SymbolInfo, Lets []LetInfo}.
- CallInfo{Name, IsPlugin, Pos, EndPos, Args []ArgInfo, FreeForm} — one per plugin call in a FROM clause and one per fn(...) call in an expression.
- SymbolInfo{Name, Pos, EndPos} — bare identifiers (columns, LET refs).
- LetInfo{Name, Pos} — LET definitions.

This keeps the LSP layer decoupled from the grammar internals while giving it exactly what validation needs: call sites with names, args, and source spans. Distinct from the reformat-oriented visitor.CallSite — do not merge them.

Current State

- Branch: lsp-server (created on top of participle-v2-upgrade).
- Commits on lsp-server:
- bb11f145c "Add VQL language server with diagnostics" — the whole LSP milestone before the truncate-and-retry experiment.
- 48b513d1d "Add artifact registry, pull diagnostics and lifecycle tests to VQL LSP" — artifact store, LSP 3.17 pull diagnostics, and the server lifecycle + artifact unit tests.
- 5c73049e8 "Add document symbols to the VQL language server".
- 002fb2dee "Add hover and completion to the VQL language server".
- 1e7ddb593 "Document editor-client byproduct of the standard LSP protocol".
- (Hybrid mode changes — bin/lsp.go, vql/lsp/artifacts.go, vql/lsp/artifacts_test.go — are in the working tree, not yet committed.)
- LSP server command velociraptor lsp implemented and verified end-to-end over stdio (initialize, didOpen/didChange/didClose, publish diagnostics, pull diagnostics, shutdown/exit). Also has --check flag to validate a query from the command line without an LSP client. Supports an optional --datastore flag; defaults to a temp datastore and generates a fresh local config if none exists (mirroring gui/collector).
- **Hybrid mode implemented** (see "Decision: hybrid mode"): the command starts a local API server, connects as superuser, and builds the registry over gRPC via GetArtifacts. Custom artifacts stored in the datastore (e.g. artifact_definitions/Custom/Test.yaml) are resolved by the LSP — verified end-to-end: a query referencing Artifact.Custom.Test(flavor=...) validates clean, an unknown param is flagged at the exact position, and a bogus artifact is reported as "Unknown plugin". Port-collision avoidance is automatic (no flags): if 8001 is taken, a free ephemeral port is used.
- Diagnostics implemented against the real plugin/function registry built from the server scope: syntax errors (precise line/col), unknown plugins/functions, unknown keyword arguments. Verified against the actual binary with a Python LSP client — e.g.

upcase(str='x') flags str as unknown, pslist(foo=1) flags foo, unknownfunc() flags the function itself.
- Whole-document parse failures are mitigated with truncate-and-retry (see Hurdles): a bad line no longer hides diagnostics on either side of it.
- Artifact names resolve against the repository (see Artifact store): SELECT * FROM Artifact.Windows.Sys.Users() no longer reports "Unknown plugin". Artifact parameters are validated against the artifact's declared parameters (e.g. foo=1 on Windows.Sys.Users flags foo as unknown).
- Unit tests in vql/lsp/ (fake registry with a couple of plugins and functions) — 30 tests total: 10 diagnostic-engine tests (clean docs, unknown function/plugin/argument, artifact arguments, multiline positions, syntax errors, truncate-and-retry recovery), 6 server lifecycle tests (initialize capabilities, didOpen+pull, didChange updates, didClose clears, pull unknown doc, shutdown closes Done), 2 document-symbol tests, 5 hover tests, 6 completion tests, and 1 API-client registry test (BuildRegistryFromAPIClient with a gomock mock client). Run with go test ./vql/lsp/.
- Document symbols implemented: vfilter.Outline() (new exported API in the vfilter repo) walks the unexported grammar AST and produces a hierarchical outline — LET variables, queries (named after their FROM plugin), SELECT columns, and function calls — each with source positions. The LSP server maps that to textDocument/documentSymbol (LSP 3.16, hierarchical DocumentSymbol[]): let→Variable, query/function →Function, column→Field. Unaliased columns get their name extracted from the source text. Advertised via DocumentSymbolProvider: true in capabilities. Verified end-to-end over the wire (see VQL-LSP-TESTS.md).
- Hover implemented: textDocument/hover resolves the symbol/argument under the cursor against the registry and returns markdown — function doc, kind (function/plugin), aggregate flag, argument names with types, or an argument's own type. Advertised via HoverProvider: true. Verified end-to-end: hovering upcase shows its real doc ("Returns the uppercase version of a string.") and real argument string; hovering pid shows int64.
- Completion implemented: textDocument/completion returns LET variables, all registered plugins/functions/artifacts, and (when the cursor is inside call parens) the callee's argument names — filtered by the word/prefix before the cursor. Kind mapping: functions→Function, variables→Variable, arguments→Field. Advertised via CompletionProvider with trigger characters . and (. Verified end-to-end: psl→pslist, upc→upcase, Art→the full artifact tree, LET vars, and pid inside pslist(.
- Agent integration verified: opencode LSP config starts the server when a .vql file is opened and returns diagnostics as agent feedback; the validate-vql custom tool (see VQL-LSP-USAGE.md Option 2) is loaded inside the live agent and returns structured diagnostics for bad queries and valid: true for clean ones.
- Possible future extensions (not currently planned): go-to-definition, and validating artifact parameters against artifact defaults.

Open Questions

- Which plugins/functions to register in the LSP scope — all server-side plugins, or a curated subset? (Resolved for now: everything in the base scope plus all artifacts from the repository. Only worth revisiting if registry size ever becomes a startup-cost problem.)
- Artifact validation depth: only artifact *names* and parameter *names* resolve. The artifact Call() implementation validates unknown parameters at runtime against artifact defaults (parameters: env) and logs "Unknown parameter %s provided to artifact" — but that check happens inside the plugin at execution time, not statically. Wiring it into the LSP registry was considered and deliberately not built: for the agent pre-flight use case, name/type checking is enough, and deep value

validation is a possible future extension, not planned work.

Artifact store (implemented)

Resolving artifact names (e.g. `Artifact.Windows.Sys.Users()`) requires loading the artifact repository into the LSP scope — the base server scope only registers built-in plugins and functions, not artifacts.

Two implementations exist in `vql/lsp/artifacts.go`:

1. **`BuildRegistryWithArtifacts(ctx, config_obj, scope)`** — loads the global artifact repository directly via `startup.StartToolServices` + `services.GetRepositoryManager` → `GetGlobalRepository` → `repository.List/Get`. Used by the unit tests and the earlier non-hybrid builds.
2. **`BuildRegistryFromAPIClient(ctx, api_client, scope)`** — fetches artifacts over the gRPC API via `GetArtifacts` (the same endpoint the GUI uses). Used by the hybrid `lsp` command, so the registry matches what the repository actually contains (including custom artifacts).

Both register each artifact as a plugin named `Artifact.<Name>` with its `parameters:` section as the argument list (`AddArtifact`). Registry population stays separate from validation logic, exactly as predicted — validation is map-lookup driven, so artifacts just add entries to the same map.

This is genuinely valuable already for diagnostics: an agent referencing a non-existent artifact (or a hallucinated artifact name) now gets an "Unknown plugin" diagnostic instead of silence, and `velociraptor vql list` alone can't do that. It becomes even more valuable once hover/completion are built (suggesting `Artifact.*` names, showing artifact docs, jumping to artifact definitions). Document symbols already benefit: the query outline is named after the FROM plugin, so `Artifact.Windows.Sys.Users` appears directly in the outline.

Artifact parameter *types* are carried through (name + type string from `ArtifactParameter`), so future type-mismatch diagnostics can use them.